

Day_13_QAOA

September 21, 2026

Week_03_Day_13_QAOA;RAJIV KUMAR YADAV

```
[1]: import numpy as np

from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
from scipy.optimize import minimize

# =====
# 1. MAX-CUT PROBLEM
# =====

# Triangle graph
edges = [
    (0, 1),
    (1, 2),
    (0, 2)
]

n = 3

# =====
# 2. CALCULATE CUT VALUE
# =====

def cut_value(bitstring):

    value = 0

    for i, j in edges:

        if bitstring[i] != bitstring[j]:
            value += 1

    return value
```

```

# =====
# 3. QAOA CIRCUIT
# =====

def create_qaoa_circuit(gamma, beta):

    qc = QuantumCircuit(n)

    # -----
    # Initial state |+>
    # -----

    for qubit in range(n):
        qc.h(qubit)

    # -----
    # Cost Hamiltonian
    # -----

    for i, j in edges:

        qc.cx(i, j)
        qc.rz(2 * gamma, j)
        qc.cx(i, j)

    # -----
    # Mixer Hamiltonian
    # -----

    for qubit in range(n):
        qc.rx(2 * beta, qubit)

    return qc

# =====
# 4. EXPECTATION VALUE
# =====

def objective(params):

    gamma = float(params[0])
    beta = float(params[1])

    qc = create_qaoa_circuit(gamma, beta)

```

```

qc.measure_all()

simulator = AerSimulator()

result = simulator.run(
    qc,
    shots=1024
).result()

counts = result.get_counts()

expectation = 0

for bitstring, count in counts.items():

    # Qiskit bitstrings are reversed
    bits = bitstring[::-1]

    cut = cut_value(bits)

    expectation += cut * count / 1024

# Minimize negative cut value
return -expectation

# =====
# 5. INITIAL PARAMETERS
# =====

initial_parameters = [
    0.5, # gamma
    0.5 # beta
]

# =====
# 6. CLASSICAL OPTIMIZATION
# =====

result = minimize(
    objective,
    initial_parameters,
    method="COBYLA",
    options={"maxiter": 50}
)

```

```

# =====
# 7. OPTIMAL PARAMETERS
# =====

gamma_opt = result.x[0]
beta_opt = result.x[1]

print("\n=====")
print("          QAOA RESULT")
print("=====")

print("Optimal gamma :", gamma_opt)
print("Optimal beta  :", beta_opt)

# =====
# 8. RUN OPTIMAL CIRCUIT
# =====

qc = create_qaoa_circuit(
    gamma_opt,
    beta_opt
)

qc.measure_all()

simulator = AerSimulator()

result_final = simulator.run(
    qc,
    shots=4096
).result()

counts = result_final.get_counts()

# =====
# 9. DISPLAY RESULTS
# =====

print("\nMeasurement results:")
print(counts)

# Find best solution

```

```

best_bitstring = None
best_cut = -1

for bitstring, count in counts.items():

    bits = bitstring[::-1]

    cut = cut_value(bits)

    if cut > best_cut:

        best_cut = cut
        best_bitstring = bitstring

print("\nBest solution:", best_bitstring)
print("Maximum cut:", best_cut)

print("=====")

```

=====

QAOA RESULT

=====

Optimal gamma : 1.2235897971118612

Optimal beta : 0.24396677747967405

Measurement results:

{'100': 668, '111': 22, '011': 699, '110': 685, '101': 651, '000': 28, '010': 684, '001': 659}

Best solution: 100

Maximum cut: 2

=====

[]: